

Outils & Technologies utilisés

Your Car Your Way — Projet P7 Architecture logicielle



PlantUML

v1.2024 · Open source · Java

RÔLE	Génération des 4 diagrammes UML à partir de fichiers texte (<code>.puml</code>)
DIAGRAMMES	Classes · Composants · Séquence · Déploiement
POURQUOI	Standard industrie. Les diagrammes sont du texte versionnable (Git). Reproductibles et modifiables facilement.
COMMANDE	<code>plantuml -tpng docs/uml/*.puml</code>
SORTIE	PNG haute résolution intégrés dans les PDF



WeasyPrint

v62 · Open source · Python

RÔLE	Conversion des fichiers HTML → PDF pour les deux livrables documents
POURQUOI	Permet de maîtriser le rendu à 100% en CSS pur. Les images UML (base64) sont intégrées directement dans le HTML → PDF auto-portant.
COMMANDE	<code>python3 docs/generate_pdf.py</code>
VS PANDOC	Pandoc nécessite LaTeX (lourd). WeasyPrint utilise CSS natif, plus simple et plus rapide à personnaliser.



Django 4.2 LTS

Python · Open source

RÔLE	Framework web backend principal
APPORTE	ORM · Admin auto-généré · Auth · i18n · CSRF
LTS	Support sécurité garanti jusqu'en avril 2026
VS FASTAPI	Admin et ORM intégrés = plusieurs semaines économisées



Django Channels 4

Extension Django officielle

RÔLE	Tchat temps réel via WebSocket
COMMENT	Transforme Django de WSGI → ASGI pour gérer les connexions persistantes
CONSUMER	<code>AsyncWebsocketConsumer</code> : connect · receive · disconnect
VS SOCKET.IO	Pas de second service Node.js à maintenir à côté



Redis 7

Base de données en mémoire · Open source

RÔLE



PostgreSQL 15

Open source · Base relationnelle SQL

RÔLE	Base de données principale de l'application
------	---

Channel Layer : diffuse les messages WebSocket entre les workers Daphne

AUSSI Cache de sessions, cache de requêtes fréquentes

LATENCE Ordre de la microseconde (stockage en mémoire RAM)

EN PROD AWS ElastiCache (service Redis managé sur le cloud)

POURQUOI Schéma strict · transactions ACID · support ORM Django natif · extension PostGIS (géo)

VS MONGODB L'audit a montré les problèmes concrets de données incohérentes sans schéma



Daphne

Serveur ASGI · Équipe Django Channels

RÔLE Serveur qui exécute Django en mode ASGI (HTTP + WebSocket simultanément)

POURQUOI Gunicorn (WSGI classique) ne supporte pas WebSocket — Daphne est obligatoire dès qu'on utilise Channels

EN PROD Placé derrière Nginx qui gère TLS (HTTPS/WSS) et le cache des fichiers statiques



Django REST Framework

DRF 3.14 · Open source

RÔLE API REST pour l'historique des messages et la liste des rooms

APPORTE Sérialisation JSON · pagination · authentification · documentation automatique

VS GRAPHQL REST suffisant pour des besoins bien définis, outillage plus simple et mature

QUESTIONS FRÉQUENTES DU MENTOR — RÉPONSES PRÉPARÉES

Q Pourquoi PlantUML plutôt que Draw.io ou Lucidchart ?

R PlantUML génère des diagrammes à partir de **texte simple** (fichiers `.puml`). Ça veut dire qu'ils sont **versionnables dans Git** (on voit les changements comme du code), reproductibles à l'identique, et modifiables sans ouvrir un outil graphique. C'est le choix standard en entreprise pour les projets qui évoluent. Draw.io c'est bien pour une démo rapide, mais pas adapté à un projet versionné.

Q Comment vous avez généré les PDF ?

R J'ai d'abord écrit les documents en **HTML/CSS** pour avoir un contrôle total sur le rendu (mise en page, couleurs, tableaux, cartes). Ensuite j'ai utilisé **WeasyPrint**, une bibliothèque Python qui convertit HTML → PDF en respectant les standards CSS. Les diagrammes UML (PNG) sont encodés en base64 et intégrés directement dans le HTML, ce qui rend les PDF totalement auto-portants — pas de fichier externe manquant.

Q Pourquoi Django Channels plutôt qu'une autre solution pour le tchat ?

R Parce que c'est l'extension **officielle** de Django pour les WebSockets. Elle s'intègre sans friction — pas besoin d'un second service (pas de Node.js à maintenir à côté). Elle transforme Django de WSGI (synchrone, pas de WS) en ASGI (asynchrone, supporte WS). Redis joue le rôle de "standard téléphonique" qui synchronise les messages entre plusieurs workers en parallèle.

Q C'est quoi ASGI et pourquoi c'est différent de WSGI ?

R WSGI (ancien standard) : le serveur traite une requête, attend la réponse, ferme la connexion. Une connexion = une requête = une réponse. **ASGI** (nouveau standard) : la connexion peut rester ouverte indéfiniment, le serveur peut envoyer des données à tout moment sans que le client l'ait demandé. C'est ce qui permet le WebSocket. Daphne est le serveur ASGI officiel de l'équipe Django Channels.

Q Pourquoi PostgreSQL et pas MySQL ou MongoDB ?

R L'audit a montré que MongoDB sans schéma strict a créé des **incohérences de données** dans l'application nord-américaine existante. PostgreSQL impose un schéma, garantit les **transactions ACID** (si une réservation échoue à mi-chemin, la base revient à son état antérieur — pas de données corrompues), et a un support Django ORM natif et premium. MySQL aurait aussi fonctionné mais PostgreSQL a de meilleures extensions (PostGIS pour la géolocalisation future).

Q Pourquoi le champ author est nullable dans Message ?

R C'est une décision de conformité **RGPD**. Quand un client supprime son compte, on ne peut pas supprimer ses messages (l'historique de la conversation reste utile pour le support en cas de litige). On applique alors le **SET_NULL** de Django : le message est conservé, mais son auteur devient NULL et s'affiche "Utilisateur supprimé" dans l'interface. C'est de l'**anonymisation** plutôt que de la suppression.